Faire « oublier » des données à un réseau de neurones : pourquoi une méthode courante s'effondre

Rapport court

Khalil Bibi
CentraleSupélec

13 octobre 2026

En bref.

— J'ai étudié comment on fait « oublier » une partie de ses données d'entraînement à un petit réseau de neurones de reconnaissance d'images (Fashion-MNIST).

— Une méthode courante, NegGrad+, s'est **effondrée** (précision du niveau du hasard) dans 26 essais sur 55, uniquement lorsque les données à oublier venaient toutes de la même classe.

— Des expériences complémentaires montrent que le problème est lié à la façon dont les couches de normalisation (*BatchNorm*) traitent ces lots d'exemples tous semblables. Deux corrections simples ont évité l'emballement dans les 30 cas concentrés testés.

— Ces résultats concernent la **stabilité** de l'optimisation. Ils ne montrent pas que les données ont réellement été effacées.

1 Le problème

Un modèle d'apprentissage automatique retient quelque chose de chaque exemple sur lequel il a été entraîné. Si l'on doit retirer certaines données (par exemple à la demande d'une personne), la solution la plus sûre est de **réentraîner le modèle sans ces données**. C'est souvent trop coûteux. Le *désapprentissage machine* (*machine unlearning*) cherche donc des méthodes plus rapides, qui modifient le modèle existant pour qu'il se comporte comme s'il n'avait jamais vu les données à oublier.

Ma question de départ était simple : **est-ce que la composition des données à oublier change le résultat ?** Oublier 600 images prises au hasard dans toutes les classes est-il différent d'oublier 600 images d'une seule classe (par exemple 600 photos de pantalons), sachant que les autres images de cette classe restent dans les données conservées ?

2 Ce que j'ai testé

— **Données :** Fashion-MNIST, 60 000 petites images de vêtements réparties en 10 classes.

— **Modèle :** un petit réseau convolutif (environ 1,7 million de paramètres) avec trois couches BatchNorm. Il atteint environ 93 % de bonnes réponses sur les images de test.

— **Deux types d'ensembles à oublier**, toujours de 600 images :

  — *aléatoire* : 60 images de chaque classe ;

  — *concentré* : 600 images d'une seule classe (une version par classe, donc 10 versions).

— **Deux méthodes d'oubli :**

— *fine-tuning* : on continue simplement l'entraînement sur les données conservées ;

— *NegGrad+* : on continue l'entraînement sur les données conservées tout en « poussant » le modèle à se tromper sur les données à oublier.

— **Référence :** pour chaque cas, un modèle réentraîné de zéro sans les données à oublier.

— **Répétitions :** 5 graines aléatoires, soit 55 essais par méthode (11 ensembles × 5 graines), sur un GPU A100 (Google Colab).

Les hypothèses, les mesures et la règle de décision de cette première étude (« v3 ») ont été écrites et datées **avant** de lancer l'expérience principale (fichier daté du 12 octobre 2026 à 22 h 51 UTC). Tout ce qui suit la section 3 a été décidé **après** avoir vu les résultats : ce sont des analyses exploratoires.

3 Ce qui s'est passé

Fine-tuning. Aucune différence détectable entre oubli aléatoire et oubli concentré. Le modèle est resté entre 90,0 % et 93,2 % de précision dans les 55 essais.

NegGrad+. Le critère fixé à l'avance était satisfait, mais pour une mauvaise raison : le modèle était **cassé**. Dans 26 essais sur 55, sa précision est tombée à environ 10 %, c'est-à-dire le niveau d'un tirage au hasard parmi 10 classes (tableau 1). Cela n'est arrivé qu'avec des ensembles concentrés, jamais avec l'ensemble aléatoire.

Tableau 1 – Nombre d'essais NegGrad+ effondrés (précision ≤ 20 %) sur 5 graines, par ensemble à oublier.

Ensemble	aléa.	c0	c1	c2	c3	c4	c5	c6	c7	c8	c9
Effondrés	0	0	**5**	**5**	0	**5**	**3**	**5**	**3**	0	0

J'ai donc considéré ce résultat comme un **échec de la méthode** dans ce cadre, et non comme la preuve d'un oubli efficace. Une partie de l'explication tient aussi au réglage : la procédure de choix des hyperparamètres a retenu, pour six classes, des réglages qui cassaient déjà le modèle.

4 Pourquoi ? Trois diagnostics

Un mot sur BatchNorm. Une couche BatchNorm normalise les valeurs internes du réseau à partir de la moyenne et de la variance du *lot* d'images en cours. Elle garde aussi des moyennes mémorisées, utilisées au moment du test. Avec NegGrad+ telle que je l'avais implémentée, les images à oublier passaient **seules** dans le réseau. Quand elles viennent toutes de la même classe, ces statistiques sont très particulières.

Diagnostic 1 : les moyennes mémorisées ou les poids ? Sur les 26 essais effondrés, j'ai gardé exactement les mêmes poids et changé seulement les moyennes mémorisées de BatchNorm. La précision médiane passe de 10 % à 69–74 %. Les moyennes jouent donc un grand rôle dans la mesure, mais les poids eux-mêmes sont abîmés (le modèle de départ est à 93 %). Un contrôle où les images à oublier passent dans le réseau mais ne comptent pas dans la perte ne s'est jamais effondré (0 sur 55). Le seul fait de faire passer ces images ne suffit donc pas à casser le modèle.

Diagnostic 2 : changer la façon de faire passer les images. J'ai comparé trois versions, sur les mêmes lots d'images :

— **A**, la version d'origine (images à oublier passées seules) ;

— **B**, images conservées et images à oublier passées *ensemble* dans un seul lot ;

— **C**, BatchNorm figée (mode évaluation) pendant l'oubli.

Sur les six classes concernées, A s'est emballée (perte d'oubli supérieure à 10) dans **29 cas sur 30**, B et C dans **aucun** (0 sur 30). B et C ont gardé entre 91,8 % et 93,3 % de précision.

Diagnostic 3 : quelles statistiques utiliser ? Sur la classe c1 (qui casse toujours) et la classe c8 (qui ne casse jamais), j'ai testé trois autres façons de normaliser les images à oublier :

— avec les moyennes mémorisées du moment (**E1**) : aucun emballement ;

— avec les statistiques du lot, mais sans laisser passer le gradient à travers elles (**E1-détaché**) : l'emballement est beaucoup plus faible (maximum de 11 à 34, contre 190 000 à 300 000 pour la version d'origine) mais le seuil est quand même franchi, et le modèle perd beaucoup en précision ;

— avec les moyennes figées du modèle de départ (**E1-O**) : emballement, y compris sur la classe témoin c8.

La figure 1 montre ces trajectoires pour une graine.

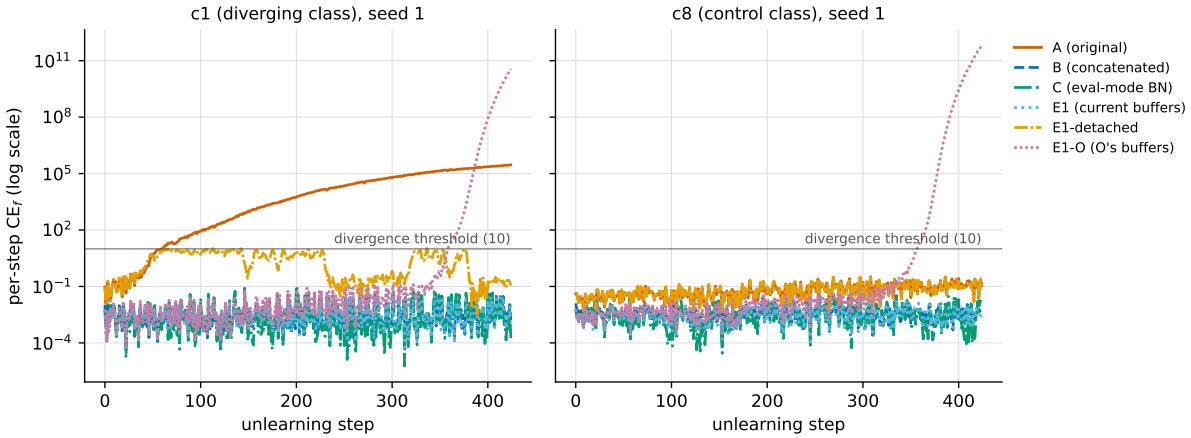


Figure 1 – Perte sur les images à oublier pendant l'oubli (échelle logarithmique), graine 1, pour la classe c1 (à gauche) et la classe témoin c8 (à droite). La ligne grise marque le seuil d'emballement (10). La version d'origine A (orange) s'emballe sur c1 ; B, C et E1 restent stables ; E1-O s'emballe tard, même sur c8. Sur c8, la courbe de A est cachée sous celle de E1-détaché.

Quand l'emballement commence. Dans la version d'origine, l'emballement démarre à un moment différent selon la classe : entre le 25e et le 37e pas (sur 425) pour c7, entre le 55e et le 89e pour c1, et après le 180e pour c5 et c6 (figure 2). Cette lecture a été faite après coup.

5 Ce qu'on peut dire, et ce qu'on ne peut pas dire

On peut dire : dans ce petit réseau, NegGrad+ telle qu'implémentée (images à oublier passées seules, BatchNorm en mode entraînement) devient instable lorsque toutes les images à oublier viennent d'une même classe. Faire passer les images ensemble (B) ou figer BatchNorm (C) a évité l'emballement dans tous les cas concentrés testés.

On ne peut pas dire :

— que les données ont été réellement effacées : je n'ai fait aucun test de confidentialité (comme une attaque par inférence d'appartenance) ;

— que les versions stables oublient bien : sur les images à oublier, elles restent plus proches du modèle de départ que du modèle réentraîné, un peu comme le simple fine-tuning ;

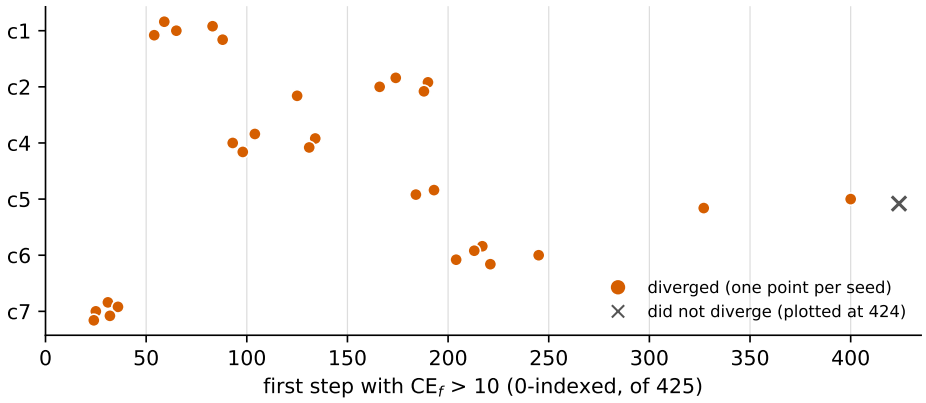


Figure 2 – Version d'origine A : premier pas où la perte d'oubli dépasse 10, par classe (un point par graine ; la croix est le seul cas, c5 graine 4, qui n'a pas franchi le seuil).

— que le mécanisme exact est identifié : ce sont des associations observées sous des interventions précises, pas une preuve de cause générale.

6 Limites

— Un seul petit réseau, un seul jeu de données, une seule taille d'ensemble à oublier (600), 5 graines.

— Les réglages ont été choisis sur une seule graine, et l'ensemble aléatoire n'a pas utilisé le même réglage de NegGrad+ que les ensembles concentrés.

— Les diagnostics ont été conçus après avoir vu les premiers résultats, et les classes étudiées ont été choisies en conséquence.

— Un seul modèle réentraîné par cas : la variabilité du réentraînement n'est pas mesurée.

— Les seuils (perte > 10, précision $\leq 20\,\%$) sont des conventions pratiques.

Code, résultats, protocoles datés et données brutes :
`https://github.com/khalilb1929/machine-Unlearning`.
Un manuscrit technique plus détaillé (en anglais) se trouve dans le même dépôt.